

DeepLabV3+ v7

2.5D Multi-View Sismik Fasiyes Segmentasyonu

Kodun tamamı + satır satır Türkçe açıklama

Özellik	v6	v7
Çözünürlük	256×256	320×320
Mixup	Yok	Var (alpha=0.2)
Label Smoothing	Yok	Var (eps=0.1)
Loss	0.5*Focal + 0.5*Dice	0.4*LS-CE + 0.3*Dice + 0.3*Focal
Batch size	8	6 (VRAM)
Encoder	EfficientNet-B4	EfficientNet-B4 (aynı)
in_channels	1 (v5'te 1ch)	3 (2.5D — tam uyumlu)

BÖLÜM 0 — Kurulum ve Cihaz Seçimi

CELL 2 — Paket kurulumu

KOD:

```
import subprocess, sys
pkgs = [
    "matplotlib",
    "scikit-learn",
    "albumentations",
    "segmentation-models-pytorch",
]
subprocess.check_call([sys.executable, '-m', 'pip', 'install', '--quiet'] + pkgs)
print("Kurulum tamamlandı.")
```

SATIR SATIR AÇIKLAMA:

subprocess.check_call([sys.executable, '-m', 'pip', 'install', ...]): Python'un kendi pip'ini çağırıyor. sys.executable sayesinde hangi Python ortamındaysak (conda, venv, sistem) oraya yüklüyor. Yanlış ortama yükleme riskini sıfırlıyor.

Kütüphane	Görevi	Neden bu, başkası değil?
matplotlib	Grafik çizimi	Standart; seaborn gibi ek bağımlılık yok
scikit-learn	confusion_matrix()	Tek fonksiyon için; kendi yazmak hatalı olurdu
albumentations	Hızlı augmentasyon	OpenCV tabanlı → numpy/torch ile sorunsuz; torchvision'dan daha hızlı
smp (segmentation-models-pytorch)	DeepLabV3+ hazırlar sunar	Encoder+decoder seçimini tek satıra indirgeyerek kod sadeleştiriyor

CELL 3 — Import ve global ayarlar

KOD — Import bloğu:

```
import os, sys, random, json, time, zipfile
from pathlib import Path
from urllib.request import urlretrieve

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, ConcatDataset, WeightedRandomSampler
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import matplotlib.patches as mpatches
from sklearn.metrics import confusion_matrix
import segmentation_models_pytorch as smp
import albumentations as A
```

AÇIKLAMA:

matplotlib.use('Agg'): GUI ekranı olmayan ortamlarda (Kaggle, sunucu, SSH) grafik çizmek için 'Agg' backend'i seçilmesi şart. Yoksa 'display not found' hatası alınıyor. Agg = Anti-Grain Geometry; dosyaya PNG kaydeder, ekrana açmaz.

torch.nn.functional as F: F.cross_entropy, F.interpolate, F.one_hot, F.softmax gibi saf fonksiyonlar burada. nn modülünden farklı: bunlar state (parametre) tutmuyor, sadece hesaplama yapıyor.

WeightedRandomSampler: Her veri noktasına farklı seçilme olasılığı verir. Sınıf dengesizliği için kritik. DataLoader'ın shuffle'ı yerine bu kullanılır.

KOD — Seed sabitleme:

```
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

Neden 4 ayrı seed? Python'un random, NumPy ve PyTorch (CPU + GPU) ayrı rastgele sayı üreteçleri var. Hepsini aynı tohumla başlatmazsak augmentasyon, veri karıştırma veya artırık başlangıç her çalıştırmada farklı olabilir. SEED=42 geleneksel; herhangi bir sayı olabilirdi.

KOD — Cihaz seçimi:

```
if torch.cuda.is_available():
    device = torch.device("cuda")
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
elif hasattr(torch.backends, 'mps') and torch.backends.mps.is_available():
    device = torch.device("mps")
    print("Apple Silicon MPS backend aktif")
else:
    device = torch.device("cpu")
    print("UYARI: GPU bulunamadı, CPU kullanılıyor!")
print(f"Cihaz: {device} | PyTorch: {torch.__version__} | SMP: {smp.__version__}")
```

torch.cuda.get_device_properties(0).total_memory / 1e9: GPU'nun toplam VRAM'ını byte cinsinden alıp GB'a çeviriyoruz. Bu bilgi batch size ve çözünürlük kararlarında önemli. Bizim RTX 4070'in 12 GB VRAM'ı var; bu yüzden 320x320 + batch=6 seçildi.

hasattr(torch.backends, 'mps'): Eski PyTorch sürümlerinde mps attribute'u hiç yok. hasattr kontrolü AttributeError'ı önüyor.

KOD — Dizin yapısı:

```
NOTEBOOK_DIR = Path(".").resolve()
PROJECT_DIR = NOTEBOOK_DIR
DATA_DIR = PROJECT_DIR / "data"
CHECKPOINTS = PROJECT_DIR / "checkpoints_v6"
FIGURES_DIR = PROJECT_DIR / "results_v6" / "figures"
METRICS_DIR = PROJECT_DIR / "results_v6" / "metrics"

for d in [DATA_DIR, CHECKPOINTS, FIGURES_DIR, METRICS_DIR]:
    d.mkdir(parents=True, exist_ok=True)
```

Path('/') operatörü: Python'un pathlib'i / operatörünü dizin birleştirmek için override etmiş. PROJECT_DIR / 'data' → '/kaggle/working/data' gibi. OS bağımsız çalışır (Windows \ sorununu çözer).

mkdir(parents=True, exist_ok=True): parents=True: Ara dizinler yoksa onlar da oluştur. exist_ok=True: Dizin zaten varsa hata verme. Bu iki parametre olmadan 'File exists' veya 'No such directory' hatası alırsın.

■ CHECKPOINTS = 'checkpoints_v6' yazıyor ama dosya adları v7. Notebook'ta küçük bir versiyon tutarsanız var — ilerlevsel olarak sorun yaratmıyor.

BÖLÜM 1 — Veri Yükleme — F3 Block Netherlands

KOD:

```
TRAIN_SEISMIC = DATA_DIR / "train" / "train_seismic.npy"

if not TRAIN_SEISMIC.exists():
    zip_path = DATA_DIR / "data.zip"
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    print("Veri indiriliyor (~1 GB)...")

def _progress(block, block_size, total):
    downloaded = block * block_size
    if total > 0:
        pct = min(100, downloaded * 100 / total)
        print(f"\r {pct:.1f}% ({downloaded/1e6:.0f} MB)", end="", flush=True)

urlretrieve("https://zenodo.org/record/3755060/files/data.zip",
            zip_path, reporthook=_progress)
print("\nCikartiliyor...")
with zipfile.ZipFile(zip_path, "r") as zf:
    zf.extractall(DATA_DIR)
    zip_path.unlink()
    print("Hazır.")
else:
    print("Veri zaten mevcut.")

print("NPY yukleniyor...")
train_seismic = np.load(DATA_DIR / "train" / "train_seismic.npy")
train_labels = np.load(DATA_DIR / "train" / "train_labels.npy")
test1_seismic = np.load(DATA_DIR / "test_once" / "test1_seismic.npy")
test1_labels = np.load(DATA_DIR / "test_once" / "test1_labels.npy")
test2_seismic = np.load(DATA_DIR / "test_once" / "test2_seismic.npy")
test2_labels = np.load(DATA_DIR / "test_once" / "test2_labels.npy")
```

SATIR SATIR AÇIKLAMA:

if not TRAIN_SEISMIC.exists(): Veri zaten indirildiyse tekrar indirme. Kaggle'da notebook her restart'ta silinir ama input dizininden çalışıyorsa zaten var. Bu kontrol ~1 GB indirmeyi önüyor.

_progress(block, block_size, total): urlretrieve'in her veri bloğunda çağrılan callback fonksiyon. \r (carriage return) imleci satır başına götürür; böylece yüzde her seferinde güncellenir, ekrana yüzlerce satır basılmaz.

zip_path.unlink(): ZIP dosyasını siliyor. NPY dosyalar çıkarıldıktan sonra ZIP'e gerek kalmıyor. Kaggle'da disk alan kıstı (20 GB), bu yüzden temizlemek önemli.

np.load(...npy): NumPy'nin kendi binary formatı. pickle olmadan saf dizi verisi — çok hızlı yükler. train_seismic shape: (401, 701, 255) → 401 inline × 701 crossline × 255 derinlik örneği.

Eksen	Boyut	Fiziksel Anlam
Eksen 0 (dim 0)	401	Inline sayıları — sismik kaynaktan alınan yöne
Eksen 1 (dim 1)	701	Crossline sayıları — inline'a dik yön

Eksen 2 (dim 2)	255	Derinlik örnekleri — yüzeyden derine do■ru
-----------------	-----	--

✓ Test1 sadece 100 inline (401→100), Test2 sadece 100 crossline (701→100) içeriyor. Bu yüzden modelin her iki yönde de iyi olmas■ gerekiyor — tek yön iyi ≠ gerçekten iyi.

BÖLÜM 2 — Keşifci Veri Analizi (EDA)

KOD:

```
CLASS_NAMES = ["Upper NS", "Lower NS", "Rijnland", "Scruff", "Zechstein", "Under Zech"]
NUM_CLASSES = 6
FACIES_COLORS = ["#3288bd", "#66c2a5", "#abdda4", "#e6f598", "#fdae61", "#f46d43"]
cmap_facies = mcolors.ListedColormap(FACIES_COLORS)

counts = np.bincount(train_labels.flatten(), minlength=NUM_CLASSES)
total = counts.sum()

print("Sinif dagilimi (Train volume):")
for i, (name, cnt) in enumerate(zip(CLASS_NAMES, counts)):
    print(f" S{i} {name:22s}: {cnt:>12,} ({cnt/total*100:.2f}%)")

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

bars = axes[0,0].bar(range(NUM_CLASSES), counts / 1e6, color=FACIES_COLORS, edgecolor="k",
    lw=0.5)
axes[0,0].set_xticks(range(NUM_CLASSES))
axes[0,0].set_xticklabels([f"S{j}\n{CLASS_NAMES[j]}" for j in range(NUM_CLASSES)], fontsize=8)
axes[0,0].set_ylabel("Piksel (Milyon)")
axes[0,0].set_title("Sinif Dagilimi")

idx_inl = train_seismic.shape[0] // 2
axes[0,1].imshow(train_seismic[idx_inl].T, cmap="seismic", aspect="auto", vmin=-1, vmax=1)
axes[0,1].set_title(f"Inline #{idx_inl}")

idx_xln = train_seismic.shape[1] // 2
axes[1,0].imshow(train_seismic[:, idx_xln, :].T, cmap="seismic", aspect="auto", vmin=-1,
    vmax=1)
axes[1,0].set_title(f"Crossline #{idx_xln}")

axes[1,1].imshow(train_labels[idx_inl].T, cmap=cmap_facies, vmin=-0.5, vmax=5.5, aspect="auto")
axes[1,1].set_title(f"Fasiyes - Inline #{idx_inl}")

plt.suptitle("F3 Block - Inline ve Crossline Kesitler", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig(FIGURES_DIR / "eda_multiview.png", dpi=150, bbox_inches="tight")
plt.show()
```

SATIR SATIR AÇIKLAMA:

mcolors.ListedColormap(FACIES_COLORS): 6 renkten özel bir colormap üretir. vmin=-0.5, vmax=5.5 ile birlikte kullanıldığında her tam sayı etiket (0,1,2,3,4,5) tam ortasına denk gelir ve renkler doğru şekilde atanır.

train_labels.flatten(): 3D array'i tek boyutlu hale getirip np.bincount'a veriyor. bincount her tam sayının kaç kez geçtiğini sayar — piksel frekansı böyle hesaplanıyor.

train_seismic[idx_inl].T: idx_inl = 200 (orta inline). train_seismic[200] shape (701, 255). .T = transpose → (255, 701). Görüntüde X eksenini crossline, Y eksenini derinlik olsun diye.

train_seismic[:, idx_xln, :].T: Tüm inline'ların aynı crossline noktasındaki kesiti. Shape: (401, 255) → transpose (255, 401).

cmap='seismic': Sismik amplitude için standart renk haritası. Negatif → mavi, sıfır → beyaz, pozitif → kırmızı. vmin=-1, vmax=1 sınırları normalize edilmiş değerlere göre.

aspect='auto': matplotlib varsayılan olarak pikselleri kare gösterir. Auto ile piksel en/boy oranı ekrana sınırsız diye ayarlanır. Sismik kesit için doğru; yoksa görüntü çok dar veya çok geniş görünür.

BÖLÜM 3 — Blok-Bazlı Train/Val Split + Sınıf Ayrılmaları

KOD:

```
n_inlines = train_seismic.shape[0] # 401
n_crosslines = train_seismic.shape[1] # 701

val_ratio = 0.2
val_start = int(n_inlines * (1 - val_ratio)) # int(401*0.8) = 320

train_inline_idx = np.arange(0, val_start) # [0, 1, ..., 319]
val_inline_idx = np.arange(val_start, n_inlines) # [320, ..., 400]
train_xline_idx = np.arange(0, n_crosslines) # [0, 1, ..., 700]

print(f"Train inline : [0, {val_start}) -> {len(train_inline_idx)} slice")
print(f"Val inline : [{val_start}, {n_inlines}) -> {len(val_inline_idx)} slice")
print(f"Train xline : [0, {n_crosslines}) -> {len(train_xline_idx)} slice")
print(f"Toplam train : {len(train_inline_idx) + len(train_xline_idx)} slice")
```

AÇIKLAMA:

Neden blok-bazlı? Sismik veri uzaysal olarak sürekli — komşu iki slice neredeyse aynı bilgiyi taşır. Rastgele seçseydin train'deki 150. ile val'deki 151. slice yan yana olurdu. Model 'komşusunu' görüyor olurdu. Buna spatial leakage denir. Blok split ile train [0-319] ve val [320-400] tamamen ayrılıyor.

Neden val'de crossline yok? Test2 crossline yönünde. Crossline'lar val'e alırsak aynı crossline hem val'de hem test2'de görünür — veri sızıntısı. Zaten gerçek değerlendirme test1 ve test2 üzerinden yapılıyor.

train_xline_idx tüm 701 crossline'ı kapsıyor. Bu crossline'ların bir kısmı val inline bölgesinden [320-400] geçiyor — minor spatial leak. Ama test verisi tamamen farklı bir volume olduğu için gerçek değerlendirme etkilenmiyor.

KOD — Focal alpha hesaplama:

```
train_counts = np.bincount(train_labels[:val_start].flatten(),
minlength=NUM_CLASSES).astype(float)
class_freq = train_counts / train_counts.sum()
focal_alpha = 1.0 / (class_freq + 1e-8)
focal_alpha = focal_alpha / focal_alpha.sum()
focal_alpha_t = torch.FloatTensor(focal_alpha).to(device)

print("\nSınıf frekansları ve Focal alpha:")
for i, (name, f, a) in enumerate(zip(CLASS_NAMES, class_freq, focal_alpha)):
    print(f" S{i} {name:22s}: freq={f:.4f} alpha={a:.4f}")
```

train_labels[:val_start]: Sadece train inline bölgesinin [0-319] etiketleri kullanılıyor. Val bölgesinin piksel frekansını kullanmak veri sızıntısı olur.

1.0 / (class_freq + 1e-8): Her sınıfın frekansının tersini alıyoruz. Örnek: S4 frekansı 0.003 ise alpha=333. S0 frekansı 0.4 ise alpha=2.5. Nadir sınıf → büyük alpha → Focal Loss'ta daha büyük ceza.

1e-8 epsilon: Eğer bir sınıf hiç yoksa class_freq=0 → 1/0 = sonsuz. 1e-8 ekleyince 1/0.00000001 = 100000000 olur ama bu normalize ediliyor. Sıfıra bölme hatası önleniyor.

focal_alpha / focal_alpha.sum(): Normalize et: toplam 1 olsun. Bu sayede loss ölçerini stabil kalıyor.

BÖLÜM 4 — 2.5D Dataset + Multi-View DataLoader

Hiperparametreler

KOD:

```
IMG_SIZE = (320, 320)
BATCH_SIZE = 6 # 320x320 + 3ch + EfficientNet-B4 için VRAM
ACCUM_STEPS = 4 # efektif batch = 24
MIXUP_ALPHA = 0.2 # Beta dağılımından lambda örneklenir
```

Neden 320x320? v6'da 256x256 kullanılıyordu. Daha yüksek çözünürlük → daha fazla spatial detay → daha iyi segmentasyon sonuçları. Ama VRAM maliyeti quadratic artıyor: $256^2 = 65K$ piksel, $320^2 = 102K$ piksel → %57 daha fazla bellek. Bu yüzden batch size 8→6'ya düürüldü.

Augmentasyon pipeline'

KOD:

```
train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.ShiftScaleRotate(shift_limit=0.1, scale_limit=0.15,
        rotate_limit=10, border_mode=0, p=0.5),
    A.ElasticTransform(alpha=80, sigma=10, p=0.3),
    A.GridDistortion(num_steps=5, distort_limit=0.2, p=0.3),
    A.RandomBrightnessContrast(brightness_limit=0.15,
        contrast_limit=0.15, p=0.4),
    A.GaussNoise(std_range=(0.001, 0.015), p=0.3),
    A.CoarseDropout(max_holes=6, max_height=20,
        max_width=20, fill=0, p=0.2),
])
```

Augmentasyon	Parametre	Fiziksel Anlam	Neden?
HorizontalFlip	p=0.5	Yatay ayna	Jeoloji yatayda simetrik olabilir
ShiftScaleRotate	shift=±10%, scale=±15%, rotate=±10°	Kaydırma+ölçek+dön dürme	Farklı görüş açıları simüle et
ElasticTransform	alpha=80, sigma=10	Yerel esnek bozulma	Sismik katmanlardaki gerçek kırılmalar
GridDistortion	5 adım, ±%20	Izgara bozulması	Ek geometrik varyasyon
RandomBrightnessContrast	±15%	Amplitüd değişimi	Farklı sismik enerji seviyeleri
GaussNoise	std 0.001-0.015	Gaussian gürültü	Gerçek sismik gürültüyü taklit et
CoarseDropout	6 delik, 20×20 px	Rastgele maske	Eksik bilgiyle çalışmayı öğren

■ VerticalFlip YOK. Derinlik eksenini fiziksel anlam taşıyor — sismik katmanlar üstte, derin katmanlar altta. Ters çevirince jeolojik anlam bozulur.

F3Dataset25D sınıfı — 2.5D input üretimi

KOD — `__init__` ve `_get_slice`:

```
class F3Dataset25D(Dataset):
# 2.5D Dataset: 3 komsu slice -> 3 kanallı input.
# axis=0 -> inline slicing, axis=1 -> crossline slicing.
def __init__(self, volume, labels, indices, axis=0, img_size=IMG_SIZE,
transform=None, augment_polarity=False):
self.volume = volume
self.labels = labels
self.indices = indices
self.axis = axis
self.n_slices = volume.shape[axis] # bu eksenin toplam slice sayısı
self.img_size = img_size
self.transform = transform
self.augment_polarity = augment_polarity

def _get_slice(self, vol, idx):
if self.axis == 0:
return vol[idx] # shape: (701, 255) – inline
else:
return vol[:, idx] # shape: (401, 255) – crossline
```

self.n_slices = volume.shape[axis]: axis=0 için n_slices=401, axis=1 için n_slices=701. Kenar kontrolü için (i_prev ve i_next klipleme) bu sayıya ihtiyaç var.

vol[:, idx] (crossline): İkinci eksen sabit, diğerleri tüm elemanlar. (401, 255) boyutunda 2D dilim alınıyor.

KOD — `__getitem__` (2.5D slice üretimi):

```
def __getitem__(self, idx):
i = self.indices[idx]
i_prev = max(0, i - 1) # ilk slice için önceki = kendisi
i_next = min(self.n_slices - 1, i + 1) # son slice için sonraki = kendisi

s_prev = self._get_slice(self.volume, i_prev).astype(np.float32)
s_curr = self._get_slice(self.volume, i).astype(np.float32)
s_next = self._get_slice(self.volume, i_next).astype(np.float32)
img = np.stack([s_prev, s_curr, s_next], axis=-1) # (H, W, 3)
mask = self._get_slice(self.labels, i).astype(np.int64)
```

max(0, i-1) ve min(n_slices-1, i+1): Kenar durumu yönetimi. i=0 ise i_prev=0 (kendisi). i=400 ise i_next=400 (kendisi). Kenar slice'larda kanal tekrarlanıyor ama bu kabul edilebilir.

np.stack([s_prev, s_curr, s_next], axis=-1): 3 tane (H, W) arrayi son ekseninde yığıyor → (H, W, 3). Albumentations (H, W, C) formatı bekliyor. Bu format ayrıca ImageNet-pretrained encoder ile uyumlu (RGB gibi düşün).

mask.astype(np.int64): PyTorch'un CrossEntropyLoss ve NLLLoss long (int64) maske bekliyor. NPY'dan float32 veya int32 gelebilir — zorla dönüştürüyoruz.

KOD — Augmentasyon + resize + tensor dönüşümü:

```

if self.augment_polarity and random.random() > 0.5:
    img = -img

if self.transform is not None:
    aug = self.transform(image=img, mask=mask.astype(np.uint8))
    img = aug["image"]
    mask = aug["mask"].astype(np.int64)

img_t = torch.from_numpy(img.copy()).permute(2, 0, 1).float() # (3, H, W)
mask_t = torch.from_numpy(mask.copy())

img_t = F.interpolate(img_t.unsqueeze(0), size=self.img_size,
mode="bilinear", align_corners=False).squeeze(0)
mask_t = F.interpolate(mask_t.float().unsqueeze(0).unsqueeze(0),
size=self.img_size, mode="nearest"
).squeeze(0).squeeze(0).long()
return img_t, mask_t

```

img = -img (polarity flip): Sismik amplitüdün işaretini akustik empedans farkına göre değiştirir. Bazı sismik ölçümler farklı polarite konvansiyonu kullanır. Bu augmentasyon modeli her iki polariteye de dayanıklıdır.

mask.astype(np.uint8) for albumentations: Albumentations maskeyi uint8 ister. 6 sınıf (0-5) uint8'e sığar. Sonra tekrar int64'e çeviriyoruz çünkü PyTorch loss long ister.

permute(2, 0, 1): $(H, W, 3) \rightarrow (3, H, W)$. PyTorch konvansiyonu: (Channels, Height, Width). NumPy/PIL (H, W, C) kullanır ama PyTorch tensörleri (C, H, W) bekler.

F.interpolate ile resize: $\text{img_t.unsqueeze(0): } (3, H, W) \rightarrow (1, 3, H, W)$ — batch boyutu ekleniyor. Sonra squeeze(0) ile geri $(3, 320, 320)$ yapıyor. Görüntü için bilinear, maske için nearest-neighbor. Maske tam sayı etiket içeriyor — bilinear ara değer üretirdi ve yanlış etiket çıkarırdı.

BÖLÜM 4b — Volume Normalizasyonu + Dataset Oluşturma

KOD:

```
# Volume normalize
train_mean = train_seismic.mean()
train_std = train_seismic.std() + 1e-8
train_seis_norm = ((train_seismic - train_mean) / train_std).astype(np.float32)
test1_seis_norm = ((test1_seismic - train_mean) / train_std).astype(np.float32)
test2_seis_norm = ((test2_seismic - train_mean) / train_std).astype(np.float32)

# Dataset'ler
train_inline_ds = F3Dataset25D(
    train_seis_norm, train_labels, train_inline_idx,
    axis=0, transform=train_transform, augment_polarity=True)

train_xline_ds = F3Dataset25D(
    train_seis_norm, train_labels, train_xline_idx,
    axis=1, transform=train_transform, augment_polarity=True)

train_ds = ConcatDataset([train_inline_ds, train_xline_ds])

val_ds = F3Dataset25D(train_seis_norm, train_labels, val_inline_idx, axis=0)
test1_ds = F3Dataset25D(test1_seis_norm, test1_labels,
    np.arange(test1_seismic.shape[0]), axis=0)
test2_ds = F3Dataset25D(test2_seis_norm, test2_labels,
    np.arange(test2_seismic.shape[1]), axis=1)
```

Neden test de train mean/std ile normalize ediliyor? Gerçek dünyada test verisi önceden görülüyor. Test'in kendi istatistiklerini kullanmak veri sızıntısı olur. Normalizasyon eğitimde öğrenilmiş bir parametre gibi düşünülmeli — inference'ta da aynı değerler kullanılmalı.

train_std + 1e-8: Standart sapma teorik olarak 0 olabilir (tüm değerler aynıysa). Bu durumda sifıra bölme olur. 1e-8 epsilon bunu önüyor.

ConcatDataset([train_inline_ds, train_xline_ds]): İki dataset'i sıralı birleştiriyor. train_ds[0:319] = inline, train_ds[320:1020] = crossline. WeightedRandomSampler'ın index sırasıyla uyumlu olması şart.

val_ds ve test_ds'de transform=None: Validation ve test sırasında augmentasyon yapılmaz. Augmentasyon sadece eğitim varyasyonu için. Test'te augmentasyon yaparsak her çalıştırmada farklı sonuç çıkar — kararsızdır.

axis=1 for test2: Test2 crossline yönünde. np.arange(test2_seismic.shape[1]) = [0, 1, ..., 99]. Dataset bu indeksleri crossline slice olarak okuyacak.

BÖLÜM 4c — WeightedRandomSampler + DataLoader

KOD:

```
RARE_CLASSES = [4, 5] # Zechstein, Under Zechstein
RARE_BOOST = 10.0 # bu sınıflara 10x daha fazla ağırlık

def compute_slice_weights(labels, indices, axis, rare_classes=RARE_CLASSES, boost=RARE_BOOST):
    weights = []
    for i in indices:
        sl = labels[i] if axis == 0 else labels[:, i]
        total_px = sl.size
        rare_px = sum(int((sl == c).sum()) for c in rare_classes)
        w = 1.0 + (rare_px / total_px) * boost
        weights.append(w)
    return weights

w_inline = compute_slice_weights(train_labels, train_inline_idx, axis=0)
w_xline = compute_slice_weights(train_labels, train_xline_idx, axis=1)
all_weights = w_inline + w_xline # ConcatDataset sırasına uygun

sampler = WeightedRandomSampler(
    weights=all_weights,
    num_samples=len(train_ds),
    replacement=True
)
```

Formül: $w = 1.0 + (\text{rare_px} / \text{total_px}) * 10.0$ — Bir slice'te %0 nadir piksel içeriyorsa $w=1.0$ (normal ağırlık). %10 nadir piksel içeriyorsa $w=1+0.1*10=2.0$ (2x seçilme şans). %50 nadir piksel içeriyorsa $w=1+0.5*10=6.0$ (6x seçilme şans).

w_inline + w_xline: Python liste birleştirilmesi. ConcatDataset aynı sınıflarla birleştirdi, bu yüzden ağırlık listesi de aynı sınıflarla birleştirilmeli.

replacement=True: Aynı slice bir epoch içinde birden fazla kez seçilebilir. False olsaydı her slice tam olarak bir kez seçilirdi — ağırlıklı örnekleme anlamsızlaştırdı.

KOD — DataLoader:

```
NUM_WORKERS = 0 if sys.platform == "win32" else 2
pin = device.type == "cuda"

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sampler,
    num_workers=NUM_WORKERS, pin_memory=pin)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, pin_memory=pin)
test1_loader = DataLoader(test1_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS)
test2_loader = DataLoader(test2_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS)
```

NUM_WORKERS=0 on Windows: Windows'ta Python multiprocessing fork kullanımı DataLoader ile hatalı çalışır. 0 = tek işlemci (yavaş ama güvenli). Linux/Mac'te 2 worker paralel veri okur.

pin_memory=True (CUDA only): Veriyi CPU'nun 'pinned' (sabitlenmiř) belleđine koyar. Bu bellek DRAM'da page-locked alanda tutulur; GPU'ya DMA ile dođrudan transfer edilebilir. Normal belleđe g re %20-30 daha h zli CPU→GPU transfer sađlar.

sampler kullanılnca shuffle=False: sampler ve shuffle aynı anda kullanılamaz — PyTorch hata verir. Sampler zaten  rnekleme sırasını kontrol ediyor.

BÖLÜM 5 — Model — DeepLabV3+ (EfficientNet-B4, in_channels=3)

KOD:

```
model = smp.DeepLabV3Plus(
    encoder_name="efficientnet-b4",
    encoder_weights="imagenet",
    in_channels=3,
    classes=NUM_CLASSES,
    activation=None,
    encoder_output_stride=16,
    decoder_atrious_rates=(12, 24, 36),
).to(device)

n_params = sum(p.numel() for p in model.parameters())
print(f"Toplam parametre: {n_params:,}")
print(f"Encoder: EfficientNet-B4 | in_channels=3 | ASPP rates=(12,24,36)")
print(f"ImageNet pretrained agirliklar 3 kanal ile tam uyumlu!")

with torch.no_grad():
    dummy = torch.randn(2, 3, 320, 320).to(device)
    out = model(dummy)
    print(f"Cikti boyutu: {out.shape} (beklenen: [2, {NUM_CLASSES}, 320, 320])")
    del dummy, out
    if device.type == 'cuda':
        torch.cuda.empty_cache()
    print("Model hazır.")
```

encoder_name='efficientnet-b4': SMP kütüphanesi bu string ile EfficientNet-B4'ü encoder olarak yükler. Alternatifler: resnet50, resnet101, vgg16, mit_b2 vb. EfficientNet-B4 seçildi çünkü parametre bakımına doğruluk oranı yüksek.

encoder_weights='imagenet': ImageNet'te önceden eğitilmiş ağırlıklar indirilir. Sıfırdan eğitime göre çok daha hızlı yakınsama ve genellikle daha yüksek final doğruluk.

in_channels=3: 2.5D girdi: [önceki, şimdiki, sonraki] slice. ImageNet ağırlıkları 3 kanallı RGB için — direkt uyumlu. v5'te 1 kanal vardı, bu yüzden ImageNet ağırlıkları uyumsuzdu.

activation=None: Model ham logit döndürür. Loss fonksiyonları (CE, Focal) kendi içlerinde softmax uygulayarak numerik olarak stabil çalışır. Burada softmax uygulanırsa çift aktivasyon olur.

encoder_output_stride=16: Encoder çıktıları girdi boyutunun 1/16'sı. 320×320 → 20×20 feature map. Stride 8 daha yüksek çözünürlük verir ama bellek 4x artar.

decoder_atrious_rates=(12, 24, 36): ASPP modülünün üç dilation rate'i. rate=12 → 12 piksel atlayarak dilim, rate=36 → 36 piksel atlayarak dilim. Farklı ölçeklerde jeolojik yapı yakalanır. Büyük rate → geniş bakış, küçük rate → ince detay.

torch.no_grad() dummy test: Model doğru şekilde çıktı ürettiği mu diye kontrol. Gradient hesaplanmıyor, sadece forward pass. del dummy + empty_cache() ile GPU belleği serbest bırakılıyor.

BÖLÜM 6 — Loss Fonksiyonlar + Mixup + Optimizer + Scheduler

Mixup fonksiyonu

KOD:

```
def mixup_data(x, y, alpha=0.2):
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1.0
    batch_size = x.size(0)
    index = torch.randperm(batch_size, device=x.device)
    mixed_x = lam * x + (1 - lam) * x[index]
    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam
```

np.random.beta(alpha, alpha): Beta(0.2, 0.2) dağılımı çoklukla 0.0-0.2 veya 0.8-1.0 aralığında değer üretir. Yani karışım ya birinci örneğe ($\lambda \approx 1$) ya ikinciye ($\lambda \approx 0$) yakın. Tam 50/50 karışım nadirdir — bu kasıtlı.

torch.randperm(batch_size): 0'dan batch_size-1'e kadar rastgele permütasyon. $x[index]$ ile batch içinde rastgele çiftler oluşturuluyor.

mixed_x = lam * x + (1-lam) * x[index]: İki görüntünün ağırlıklı ortalaması. Hem görüntü karıştırılıyor hem de loss hesabında $\lambda * \text{criterion}(\text{pred}, y_a) + (1-\lambda) * \text{criterion}(\text{pred}, y_b)$ kullanılıyor.

FocalLoss sınıfı

KOD:

```
class FocalLoss(nn.Module):
    def __init__(self, alpha, gamma=2.0):
        super().__init__()
        self.register_buffer('alpha', alpha)
        self.gamma = gamma

    def forward(self, pred, target):
        ce = F.cross_entropy(pred, target, reduction='none')
        pt = torch.exp(-ce)
        alpha_t = self.alpha[target]
        loss = alpha_t * ((1 - pt) ** self.gamma) * ce
        return loss.mean()
```

register_buffer('alpha', alpha): alpha tensörü parametre değil ama cihaz (CPU/GPU) ile birlikte taşınması gerekiyor. register_buffer bunu sağlıyor. model.to(device) yapınca alpha da taşınır. model.parameters() içinde görünmez → optimizer update etmez.

F.cross_entropy(..., reduction='none'): Her piksel için ayrı ayrı CE loss değerleri döndürür — vektör, skalar değil. Sonra focal ağırlık uygulanacak; önce toplama yapmamalı.

pt = torch.exp(-ce): CE = $-\log(\text{pt})$ oldu■undan $\text{pt} = \exp(-\text{CE})$. pt modelin do■ru s■n■fa atad■■■ olas■■■k. pt büyükse (model emin) $\rightarrow (1-\text{pt})^2$ küçük $\rightarrow$ loss azal■r. Kolay örnekler cezaland■r■lmaz.

alpha_t = self.alpha[target]: target tensörü index olarak kullan■■■yor. Her piksel için o piksel s■n■f■n■n alpha de■eri seçiliyor. Nadir s■n■f $\rightarrow$ büyük alpha $\rightarrow$ loss büyük.

DiceLoss s■n■f■

KOD:

```
class DiceLoss(nn.Module):
    def __init__(self, n_classes=NUM_CLASSES, smooth=1.0):
        super().__init__()
        self.n_classes = n_classes; self.smooth = smooth

    def forward(self, pred, target):
        pred_soft = torch.softmax(pred, dim=1)
        target_oh = F.one_hot(target, self.n_classes).permute(0, 3, 1, 2).float()
        dice = 0.0
        for c in range(self.n_classes):
            p = pred_soft[:, c]; t = target_oh[:, c]
            inter = (p * t).sum()
            dice += (2 * inter + self.smooth) / (p.sum() + t.sum() + self.smooth)
        return 1 - dice / self.n_classes
```

F.one_hot(target, 6).permute(0,3,1,2): target shape (B,H,W) $\rightarrow$ one_hot $\rightarrow$ (B,H,W,6) $\rightarrow$ permute $\rightarrow$ (B,6,H,W). Böylece pred_soft ile aynı shape. Her s■n■f için binary mask.

inter = (p * t).sum(): Element-wise çarp■m: model C s■n■f■ için verdi■i olas■■■k $\times$ gerçek C maskesi. Her ikisi de 1 olan pikseller kesi■imi olu■turuyor.

smooth=1.0 (Laplace): E■er bir s■n■f hiç yoksa $p.\text{sum}()=0$, $t.\text{sum}()=0$, $\text{inter}=0 \rightarrow 1/(1+0+0+1) = 0.5$. Böylece model var olmayan s■n■f için ne cezaland■r■■■yor ne ödüllendiriliyor.

TripleLoss — Ana loss fonksiyonu

KOD:

```
class TripleLoss(nn.Module):
    # v7 triple loss: 0.4 * LS-CE + 0.3 * Dice + 0.3 * Focal
    def __init__(self, alpha, gamma=2.0, label_smoothing=0.1):
        super().__init__()
        self.focal = FocalLoss(alpha, gamma)
        self.dice = DiceLoss()
        self.label_smoothing = label_smoothing

    def forward(self, pred, target):
        ls_ce = F.cross_entropy(pred, target, label_smoothing=self.label_smoothing)
        return 0.4 * ls_ce + 0.3 * self.dice(pred, target) + 0.3 * self.focal(pred, target)
```

label_smoothing=0.1: Klasik CE: do■ru s■n■f hedefi=1.0, di■erleri=0.0. Smoothed: do■ru s■n■f hedefi = $1-0.1+0.1/6 \approx 0.917$, di■erleri = $0.1/6 \approx 0.017$. Model daha az overconfident olur, kalibrasyonu iyile■ir.

Neden 0.4 LS-CE + 0.3 Dice + 0.3 Focal? LS-CE genel öğrenmeyi, Dice bölge cakırtmasını, Focal zor/nadir örnekleri optimize eder. Ağırlıklar deneysel: Focal ve Dice elit önem taşıyor, LS-CE biraz daha ağır çünkü genel sınıflandırma temel.

Optimizer ve Scheduler

KOD:

```
NUM_EPOCHS = 100
criterion = TripleLoss(focal_alpha_t, gamma=2.0, label_smoothing=0.1).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer,
    T_0=25,
    T_mult=1,
    eta_min=1e-6,
)
```

AdamW vs Adam: Adam'da weight decay gradient güncellemesinin içinde uygulanır (L2 reg ile eşdeğer değil). AdamW weight decay'i ayrı uyguluyor → daha doğru regularizasyon. Transformerlar ve EfficientNet gibi büyük modellerde AdamW tercih edilir.

weight_decay=1e-3: Her güncelleme adımında ağırlıklar 0.999 ile çarpılıyor (küçük azalma). Bu overfitting'i azaltır; ağırlıklar gereksiz yere büyümmez.

CosineAnnealingWarmRestarts (T_0=25, T_mult=1, eta_min=1e-6): lr 1e-4'ten 1e-6'ya cosine eşiğiyle düşüyor (25 epoch). 25. epochta tekrar 1e-4'e çıkıyor (restart). 50, 75, 100. epochlarda tekrar. T_mult=1 → her restart aynı süre (25 epoch). T_mult=2 deseydi 25→50→100 epoch olurdu.

BÖLÜM 7 — Metrik Fonksiyonlar

KOD:

```
def compute_metrics(preds_all, targets_all, n=NUM_CLASSES):
    res = {}
    res['PA'] = float((preds_all == targets_all).sum() / len(targets_all))
    cm = confusion_matrix(targets_all, preds_all, labels=list(range(n)))

    row_sums = cm.sum(axis=1).astype(float)
    class_acc = np.where(row_sums > 0, cm.diagonal() / row_sums, 0.0)
    res['MCA'] = float(class_acc.mean())
    res['per_class_acc'] = class_acc.tolist()

    ious = []
    for c in range(n):
        inter = cm[c, c]
        union = cm[c,:].sum() + cm[:,c].sum() - inter
        ious.append(float(inter / (union + 1e-8)))
    res['mIoU'] = float(np.mean(ious))
    res['per_class_iou'] = ious

    dices = []
    for c in range(n):
        tp = cm[c,c]; fp = cm[:,c].sum()-tp; fn = cm[c,:].sum()-tp
        dices.append(float(2*tp / (2*tp + fp + fn + 1e-8)))
    res['mean_dice'] = float(np.mean(dices))
    res['per_class_dice'] = dices
    res['confusion_matrix'] = cm.tolist()
    return res
```

PA (Pixel Accuracy): Tüm doğru piksel sayısı / toplam piksel. Hesaplaması kolay ama yanıltıcı. Eğer S0 veriyi %70 kaplıyorsa ve model sadece S0 tahmin ederse PA=%70 görünür. Bu yüzden birincil metrik olarak kullanılmaz.

cm = confusion_matrix(...): 6×6 matris. $cm[i,j]$ = gerçekte i sınıfı olan ama j tahmin edilen piksel sayısı. Köşegen = doğru tahminler. Köşegen dışı = hatalar.

np.where(row_sums > 0, cm.diagonal() / row_sums, 0.0): Bir sınıf hiç yoksa row_sum=0 → sifra bölme. np.where ile bu durumda 0.0 atanıyor.

IoU hesabı: $cm[c,:].sum() + cm[:,c].sum() - inter$: $cm[c,:].sum()$ = c sınıfı için toplam gerçek piksel (TP+FN). $cm[:,c].sum()$ = c olarak tahmin edilen toplam piksel (TP+FP). Toplarsak TP iki kez sayılır, bir kere çıkarıyoruz. $IoU = TP / (TP+FP+FN)$.

Dice: $2*TP / (2*TP + FP + FN)$: IoU ile matematikte eşdeğer ama formül farklı. $Dice = 2*IoU / (1+IoU)$. Dice genellikle IoU'dan biraz yüksek çıkar — TP'yi iki kez sayıyor.

BÖLÜM 8 — Eğitim Döngüsü — 100 Epoch + Grad Accumulation + Early Stop

Checkpoint yükleme ve AMP ayarları

KOD:

```
CHECKPOINT_PATH = CHECKPOINTS / "deeplabv3plus_v7_checkpoint.pth"
BEST_MODEL_PATH = CHECKPOINTS / "deeplabv3plus_v7_best.pth"

use_amp = device.type == "cuda"
scaler = torch.amp.GradScaler("cuda") if use_amp else None

start_epoch = 0
best_val_miou = 0.0
patience_counter = 0
PATIENCE = 25
history = {'train_loss': [], 'val_loss': [], 'val_miou': [], 'val_dice': [], 'lr': []}

if CHECKPOINT_PATH.exists():
    print("Checkpoint bulundu, devam ediliyor...")
    ckpt = torch.load(CHECKPOINT_PATH, map_location=device, weights_only=False)
    model.load_state_dict(ckpt['model_state'])
    optimizer.load_state_dict(ckpt['optimizer_state'])
    scheduler.load_state_dict(ckpt['scheduler_state'])
    start_epoch = ckpt['epoch'] + 1
    best_val_miou = ckpt['best_val_miou']
    history = ckpt['history']
    patience_counter = ckpt.get('patience_counter', 0)
    if use_amp and 'scaler_state' in ckpt:
        scaler.load_state_dict(ckpt['scaler_state'])
    print(f"Epoch {start_epoch}/{NUM_EPOCHS} - önceki en iyi mIoU: {best_val_miou:.4f}")
else:
    print("Sifirdan başlıyor...")
```

torch.amp.GradScaler('cuda'): AMP (Automatic Mixed Precision) için gradient ölçekleyici. float16'da küçük gradient değerleri underflow ile 0'a dönebilir. GradScaler loss'u büyük bir sayıyla çarpar, geri yayar, sonra optimizer update öncesinde tekrar böler.

map_location=device: Checkpoint GPU'da kaydedildiyse, CPU ortamında yüklemek için device='cpu' gerekir. map_location otomatik çözüyor.

ckpt.get('patience_counter', 0): Eski checkpoint'larda bu alan olmayabilir. get() ile varsayılan 0 kullanılıyor — KeyError yok.

start_epoch = ckpt['epoch'] + 1: Kaydedilen epoch tamamlanmış son epoch. Bir sonrakinden başlamak için +1.

Eğitim döngüsü — train adımı

KOD:

```

for epoch in range(start_epoch, NUM_EPOCHS):
    t0 = time.time()

    # --- TRAIN ---
    model.train()
    optimizer.zero_grad()
    train_loss = 0.0
    for step, (imgs, masks) in enumerate(train_loader):
        imgs, masks = imgs.to(device, non_blocking=True), masks.to(device, non_blocking=True)

        # Mixup: %50 olasilikla uygula
        use_mixup = random.random() < 0.5
        if use_mixup:
            imgs, masks_a, masks_b, lam = mixup_data(imgs, masks, MIXUP_ALPHA)

        if use_amp:
            with torch.amp.autocast("cuda"):
                preds = model(imgs)
            if use_mixup:
                loss = (lam * criterion(preds, masks_a) +
                        (1 - lam) * criterion(preds, masks_b)) / ACCUM_STEPS
            else:
                loss = criterion(preds, masks) / ACCUM_STEPS
            scaler.scale(loss).backward()
        else:
            preds = model(imgs)
            if use_mixup:
                loss = (lam * criterion(preds, masks_a) +
                        (1 - lam) * criterion(preds, masks_b)) / ACCUM_STEPS
            else:
                loss = criterion(preds, masks) / ACCUM_STEPS
            loss.backward()

        if (step + 1) % ACCUM_STEPS == 0 or (step + 1) == len(train_loader):
            if use_amp:
                scaler.unscale_(optimizer)
                torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
                scaler.step(optimizer)
                scaler.update()
            else:
                torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
                optimizer.step()
                optimizer.zero_grad()

        train_loss += loss.item() * ACCUM_STEPS
    train_loss /= len(train_loader)
    scheduler.step()

```

model.train(): Dropout ve BatchNorm'u eğitim moduna alır. BatchNorm running mean/var güncellemesi sadece train modunda olur. Inference'ta model.eval() ile dondurulur.

non_blocking=True: pin_memory=True ile birlikte: CPU→GPU transferi asenkron yapılır. GPU hesap yaparken bir sonraki batch transfer edilmeye başlanır.

loss / ACCUM_STEPS: Gradient accumulation'da her mikro-batch loss'unu ACCUM_STEPS=4'e böl. 4 adım biriktirince toplam loss orjinal batch size'n loss'una eşit olur. Böl → biriktir → güncelle mantığı.

scaler.unscale_(optimizer): Gradient'lar hâlâ ölçeklenmiş durumda. clip_grad_norm uygulamadan önce gerçek değerlerine döndürülmeli. unscale_ bunu yapıyor.

(step+1) == len(train_loader): Son batch tam 4'e bölünmeyebilir. Bu koşul son kalan adımlar da günceller. Örnek: 100 batch, ACCUM=4 → 25 güncelleme. 101 batch → 25 güncelleme + son 1 batch da güncellenir.

train_loss += loss.item() * ACCUM_STEPS: loss /ACCUM_STEPS ile ölçeklendi; geri alarak gerçek loss değerini biriktiriyoruz. Epoch sonunda len(train_loader)'a bölerek ortalama alınıyor.

scheduler.step() epoch sonunda: CosineAnnealingWarmRestarts epoch bazında step atar. Batch sonunda çağırılrsa LR yanlış hesaplanır.

Validation + Checkpoint + Early Stopping

KOD:

```
# --- VALIDATION ---
model.eval()
val_loss = 0.0
all_p, all_t = [], []
with torch.no_grad():
    for imgs, masks in val_loader:
        imgs, masks = imgs.to(device, non_blocking=True), masks.to(device, non_blocking=True)
        if use_amp:
            with torch.amp.autocast("cuda"):
                preds = model(imgs)
        else:
            preds = model(imgs)
        val_loss += criterion(preds, masks).item()
        all_p.append(preds.argmax(1).cpu().numpy().flatten())
        all_t.append(masks.cpu().numpy().flatten())
val_loss /= len(val_loader)
val_metrics = compute_metrics(np.concatenate(all_p), np.concatenate(all_t))
```

model.eval() + torch.no_grad(): eval() → BatchNorm istatistiklerini dondur, Dropout kapat. no_grad() → gradient hesaplanmaz → bellek tasarrufu + hız artışı.

preds.argmax(1): 6 kanallı logit tensörünün (B,6,H,W) maksimum değerinin indeksi. dim=1 boyutu boyunca → (B,H,W) sınıf haritası. Kimin logiti en büyükse o sınıf tahmin ediliyor.

.cpu().numpy().flatten(): GPU → CPU → NumPy → 1D array. Her batch'i flatten edip listeye ekliyoruz. Sonunda np.concatenate ile tüm piksel tahminleri tek arrayde birleştiriyor.

KOD — Checkpoint kaydet ve early stopping:


```

marker = ""
if val_metrics['mIoU'] > best_val_miou:
    best_val_miou = val_metrics['mIoU']
    torch.save(model.state_dict(), BEST_MODEL_PATH)
    marker = " *** BEST ***"
    patience_counter = 0
else:
    patience_counter += 1

ckpt_data = {
    'epoch': epoch, 'model_state': model.state_dict(),
    'optimizer_state': optimizer.state_dict(),
    'scheduler_state': scheduler.state_dict(),
    'best_val_miou': best_val_miou, 'history': history,
    'patience_counter': patience_counter,
}
if use_amp:
    ckpt_data['scaler_state'] = scaler.state_dict()
torch.save(ckpt_data, CHECKPOINT_PATH)

if patience_counter >= PATIENCE:
    print(f"\nEarly stopping – {PATIENCE} epoch boyunca iyileşme yok.")
    break

```

■ki ayr■ kay■t dosyas■: CHECKPOINT_PATH: her epoch kaydedilir, üzerine yaz■l■r (devam için). BEST_MODEL_PATH: sadece mIoU iyile■ince kaydedilir (test için kullan■l■r). ■kisi farklı. Checkpoint büyük (optimizer dahil), best sadece model a■l■rl■klar■.

optimizer_state ve scheduler_state kayd■ neden ■art? Adam optimizer her parametre için momentum ve variance geçmi■i tutar. Bunlar kaydedilmezse restart'ta optimizer s■f■rdan ba■lar — ilk birkaç epoch hatal■ güncelleme.

PATIENCE=25 early stopping: 25 epoch boyunca val mIoU iyile■mezse dur. Çok küçük patience a■l■r■ erken durur, çok büyük patience gereksiz hesap yapar. 25, CosineWarmRestarts T_0=25 ile uyumlu — tam bir döngü bekliyor.

BÖLÜM 9 — Test Zamanı Augmentasyonu (TTA)

KOD — Model yükleme + TTA fonksiyonu:

```
model.load_state_dict(torch.load(BEST_MODEL_PATH, map_location=device, weights_only=True))
model.eval()

def run_eval_tta(loader, tta=True):
    all_p, all_t = [], []
    with torch.no_grad():
        for imgs, masks in loader:
            imgs_dev = imgs.to(device, non_blocking=True)

            if use_amp:
                with torch.amp.autocast("cuda"):
                    logits = model(imgs_dev)
            else:
                logits = model(imgs_dev)
            probs = torch.softmax(logits, dim=1)

            if tta:
                # --- HFlip TTA ---
                imgs_hf = torch.flip(imgs_dev, dims=[3])
                if use_amp:
                    with torch.amp.autocast("cuda"):
                        logits_hf = model(imgs_hf)
                else:
                    logits_hf = model(imgs_hf)
                probs += torch.softmax(torch.flip(logits_hf, dims=[3]), dim=1)

                # --- Polarity TTA ---
                imgs_neg = -imgs_dev
                if use_amp:
                    with torch.amp.autocast("cuda"):
                        logits_neg = model(imgs_neg)
                else:
                    logits_neg = model(imgs_neg)
                probs += torch.softmax(logits_neg, dim=1)

            probs /= 3.0

    all_p.append(probs.argmax(1).cpu().numpy().flatten())
    all_t.append(masks.numpy().flatten())
    return np.concatenate(all_p), np.concatenate(all_t)
```

weights_only=True: Güvenlik. Sadece tensor ağırlıkları yüklenir, Python nesneleri execute edilmez. Kötü amaçlı checkpoint'lara karşı önlem. CHECKPOINT_PATH için weights_only=False çünkü optimizer state dict nesnesi içeriyor.

torch.flip(imgs_dev, dims=[3]): dims=[3] → W (geniçlik) boyutunda ters çevirme. Tensor shape (B, C, H, W): dim 0=batch, 1=kanal, 2=yükseklik, 3=geniçlik.

torch.flip(logits_hf, dims=[3]): Çok kritik! Flipped görüntüden gelen tahmin de flip'lenmeli. Aksi takdirde sol-sağ koordinatlar yer değiştirilmi olarak toplanır — anlamsız sonuç.

probs /= 3.0: 3 tahmin toplandı (orijinal + hflip + polarity). Ortalamalar alınıyor. Softmax olasılıklar toplandı ve bölündü — bu olasılıkların ortalaması. En yüksek ortalama olasılıkla sınıf final tahmin.

masks.numpy() (CPU'dan direkt): masks DataLoader'dan geliyor ve hiç GPU'ya gönderilmedi. Doğrudan .numpy() çağrılabilir. .cpu() gerekmez.

BÖLÜM 10-13 — Görselleştirme Bölümleri

Eğitim ekrani (Bölüm 10)

KOD:

```
epochs_x = range(1, len(history['train_loss']) + 1)
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].plot(epochs_x, history['train_loss'], label='Train', color='steelblue')
axes[0].plot(epochs_x, history['val_loss'], label='Val', color='coral')
axes[0].set_title('Loss'); axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_x, history['val_miou'], label='mIoU', color='green')
axes[1].plot(epochs_x, history['val_dice'], label='Dice', color='purple')
axes[1].set_title('Metrikler'); axes[1].legend(); axes[1].grid(alpha=0.3)

axes[2].plot(epochs_x, history['lr'], color='darkorange')
axes[2].set_title('Learning Rate'); axes[2].grid(alpha=0.3)

for ax in axes: ax.set_xlabel('Epoch')
plt.suptitle("DeepLabV3+ v6 – Eğitim Süreci (2.5D + EfficientNet-B4)", ...)
plt.tight_layout()
plt.savefig(FIGURES_DIR / "training_curves_v6.png", dpi=150, bbox_inches="tight")
```

history dict: Her epoch sonunda train_loss, val_loss, val_miou, val_dice ve lr kaydedildi. Checkpoint içinde de saklandıktan sonra restart'ta kaybolmaz.

Ne bakılmalı? Train loss düşüyor, val loss artıyorsa → overfitting. Val mIoU plateau yapıyorsa early stopping tetiklenebilir. LR grafiğinde her 25 epochta restart görülmeli.

Segmentasyon kararlaştırması (Bölüm 11)

KOD — Kritik satır:

```
axes[row, 0].imshow(img[1].numpy(), cmap="seismic", vmin=-2, vmax=2, aspect="auto")
# img[1] = orta kanal (şimdiki slice) – 3 kanallı inputun 'gerçek' dilimi
```

img[1] neden? img shape (3, H, W): kanal 0 önceki slice, kanal 1 şimdiki, kanal 2 sonraki. Görselleştirmede asıl slice'i göstermek için indeks 1 kullanılıyor.

Confusion Matrix (Bölüm 12)

KOD:

```
cm_arr = np.array(metrics_all['confusion_matrix'])
cm_norm = cm_arr.astype(float) / cm_arr.sum(axis=1, keepdims=True).clip(min=1)

for ax, data, title in zip(axes, [cm_arr, cm_norm], ["Ham Sayılar", "Normalize (satir %)"]):
    im = ax.imshow(data, cmap="Blues")
    ...
    for i in range(NUM_CLASSES):
        for j in range(NUM_CLASSES):
            val = f"{data[i,j]:.2f}" if title != "Ham Sayılar" else f"{int(data[i,j]):,}"
            ax.text(j, i, val, ha="center", va="center", fontsize=6,
                    color="white" if data[i,j] > data.max()*0.5 else "black")
```

cm_norm = cm / sum(axis=1, keepdims=True): Her satır kendi toplamına bölüyor. axis=1 → satır toplamları. keepdims=True → shape (6,1) olsun, (6,6) ile broadcast edilebilsin. clip(min=1) → boş satırlarda sıfıra bölmeyi önüyor.

Yazı rengi: 'white' if > max*0.5 else 'black': Koyu arka plan üzerinde beyaz, açık arka planda siyah yazı. Kontrastı koruyarak okunabilirliği artırıyor.

Versiyon Karşılaştırma (Bölüm 14)

KOD — Sabit değer fallback:

```
if v5_metrics_path.exists():
    with open(v5_metrics_path) as f:
        v5_data = json.load(f)
    # ... JSON'dan yükle
else:
    v5 = {"test1_miou": 0.7577, "test2_miou": 0.6585,
         "combined_miou": 0.7582, ...}
    print(" v5 metrikleri sabit degerlerden.")
```

Sabit değer fallback neden? v5 modeli aynı ortamda çalıştırılabilir ya da olmayabilir. JSON yoksa hard-coded değerler kullanılıyor. Bu karşılaştırma tablosunu her zaman üretebilmek için pratik bir çözüm.

■ v7 notebook'ta birkaç string hâlâ 'v6' yazıyor (plot başlıklar, klasör adları). Bu ilerleyişle değil, sadece versiyon notasyonu tutarsızlıklar.

BÖLÜM ÖZET — Tüm Pipeline — v7 Neyi Neden Yapıyor?

Adım	Teknik Karar	Gerekçe
Veri split	Blok-bazlı %80/%20	Spatial leakage engelle
Normalizasyon	Train mean/std → test'e de uygula	Veri sızıntısını önle
2.5D input	3 komşu slice → 3 kanal (320×320)	Derinlik süreklilik + ImageNet uyumu
Augmentasyon	7 teknik, VerticalFlip YOK	Gerçekçi varyasyon, jeolojik anlam koru
Sampler	WeightedRandomSampler (10× boost)	S4/S5 nadir sınıfları daha çok gör
Model	DeepLabV3+ EfficientNet-B4 ASPP(12,24,36)	Çok-ölçekli jeolojik özellik yakalama
Loss	0.4 LS-CE + 0.3 Dice + 0.3 Focal	Her sorun için ayrı loss bileşeni
Mixup	alpha=0.2, p=0.5	Regularizasyon güçlendirme (v7 yeniliği)
Optimizer	AdamW lr=1e-4 wd=1e-3	Doğru weight decay, stabil öğrenme
Scheduler	CosineWarmRestarts T_0=25	Periyodik restart ile lokal optimum kaçış
Grad Accum	4 adım → efektif batch 24	VRAM kısıtında büyük batch etkisi
AMP	float16 (CUDA)	VRAM ve hız tasarrufu
Grad Clip	max_norm=1.0	Gradient explosion önle
Checkpoint	Her epoch tam state	Kesintisiz eğitim
Early Stop	patience=25	Fazla hesaplamayı engelle
TTA	HFlip + Polarity, 3× ortalama	Bedavaya +1-3% mIoU
Metrik	mIoU birincil	Sınıf dengesizliğini gizlemez